

THE COMPLETE COURSE

Learn Roblox Studio. Ship real games.

Nine modules, 43 lessons, one shipped game: take Coin Rush from an empty baseplate to a published, monetized Roblox game.

Contents

01 Getting Started

1.1 What you can build in Roblox Studio · 1.2 Installing Studio and setting up your account · 1.3 The interface: Explorer, Properties, Toolbox · 1.4 Camera controls and your first part · 1.5 Playtesting and your first private publish

02 World Building

2.1 Parts, size, color and materials · 2.2 Anchoring and physics · 2.3 Terrain and water · 2.4 Lighting and atmosphere · 2.5 Using models safely (no backdoors)

03 Scripting Fundamentals

3.1 Script vs LocalScript vs ModuleScript · 3.2 Variables and types · 3.3 Conditions and loops · 3.4 Functions · 3.5 Events: build a lava floor · 3.6 Tables

04 Player & UI

4.1 Build a studio-grade intro screen · 4.2 ScreenGui, Frames and TextLabels · 4.3 Scaling for desktop, mobile and console · 4.4 Animation with TweenService · 4.5 Main menu and HUD

05 Client & Server

5.1 How the client-server model works · 5.2 RemoteEvents and RemoteFunctions · 5.3 Never trust the client · 5.4 Hardening your game against exploiters

06 Data & Progression

6.1 Leaderstats: coins and points · 6.2 DataStoreService · 6.3 Saving on leave and on server shutdown · 6.4 XP and levels

07 The Game Loop

7.1 Designing your game on paper · 7.2 Rounds and timers · 7.3 Collectibles and scoring · 7.4 Upgrade shop · 7.5 Sound and visual effects

08 Monetization

8.1 Game Passes · 8.2 Developer Products · 8.3 Bulletproof ProcessReceipt · 8.4 Pricing that doesn't push players away

09 Launch & Growth

9.1 Icon and thumbnails that get clicks · 9.2 Title, description and discoverability · 9.3 Analytics: finding where players drop off · 9.4 Updates and building a community · 9.5 Running Roblox Ads

MODULE 01 OF 09

01

Getting Started

YOU'LL SHIP

A playable test place on your Roblox profile.

- 1.1 What you can build in Roblox Studio
- 1.2 Installing Studio and setting up your account
- 1.3 The interface: Explorer, Properties, Toolbox
- 1.4 Camera controls and your first part
- 1.5 Playtesting and your first private publish

Every Roblox game you've ever played started as an empty baseplate in Roblox Studio. In this module you'll install Studio, learn to move around like a builder, place your first parts, and publish a private test place to your profile.

The project for this course: over the next nine modules you'll build **Coin Rush**, a round-based game where players race to collect coins, dodge lava, buy upgrades, and come back tomorrow because their progress is saved. Every lesson adds one real piece of it.

1.1 What you can build in Roblox Studio

Roblox Studio is a full game engine. The same tool is used by solo creators and by studios with dozens of developers. With it you get:

- **A 3D world** with physics, lighting, terrain and water.
- **Luau**, a fast scripting language based on Lua, with optional types.
- **Multiplayer by default.** Every game runs on Roblox servers, and players join automatically. You never set up networking hardware.
- **Built-in services:** saving data, in-game purchases, matchmaking, chat, analytics.
- **Instant distribution** to PC, Mac, phones, tablets, consoles and VR.

Most successful Roblox games fit a small set of genres: obbies (obstacle courses), tycoons, simulators, horror, roleplay, tower defense, and competitive round-based games. Coin Rush is round-based on purpose: it teaches the loop that powers almost every genre, which is *start* → *play* → *reward* → *repeat*.

TIP Play three popular games in the genre you like before you build. Write down what happens in the first 30 seconds of each. First impressions decide whether a player stays.

1.2 Installing Studio and setting up your account

1. Go to **create.roblox.com** and sign in with your Roblox account (create one if you don't have it).
2. Click **Start Creating / Download Studio** and run the installer.
3. Open Studio and sign in with the same account.
4. Turn on **2-step verification** in your Roblox account settings. You'll be publishing games and later handling Robux, so protect the account.

Studio runs on Windows and macOS. It does not run on phones, tablets or Chromebooks, although you can *test* your game on those devices later.

WATCH OUT Never sign in to Studio plugins or websites that ask for your Roblox password or cookie. No legitimate tool needs it.

1.3

The interface: Explorer, Properties, Toolbox

Create a new place from the **Baseplate** template. You'll see four areas you'll use every day:

Window	What it's for
Viewport	The 3D view of your world.
Explorer	A tree of everything in your game: parts, scripts, UI, services.
Properties	Settings of whatever you selected: size, color, material, name...
Toolbox	Free models, images, sounds and plugins from the community.

If a window is missing, open it again from the **View** (or **Window**) menu.

The Explorer is the most important one. Everything in a Roblox game is an **Instance** that lives somewhere in this tree:

- **Workspace**: everything that exists in the 3D world.
- **Players**: the people currently in the game.
- **Lighting**: sky, sun, fog, time of day.
- **ReplicatedStorage**: things both the server and players can access.
- **ServerScriptService**: scripts that only run on the server.
- **StarterGui**: UI that gets copied to each player.
- **StarterPlayer**: scripts and settings copied into each player's character.

You'll learn what each of these is for as you need it. For now, remember: *where* you put something decides *how* it behaves.

1.4

Camera controls and your first part

Camera:

- **Right mouse button + drag**: look around.
- **W A S D**: move. **Q / E**: down / up.
- **Shift** (hold): move slower for precise placement.
- **F**: focus the camera on the selected object.
- **Mouse wheel**: zoom.

Your first part:

1. In the **Home** tab click **Part**. A grey block appears.
2. Use the **Move**, **Scale** and **Rotate** tools (Home tab, or **Ctrl+2** / **Ctrl+3** / **Ctrl+4**) to shape it.
3. In **Properties**, change **BrickColor** or **Color**, and **Material** (try Neon, Wood, Metal).
4. Rename it in the Explorer (double-click the name) to `StartPlatform`.

TIP Name things as you create them. `Part (37)` means nothing in two weeks. `LavaFloor_Arena2` does.

Snapping: in the Model tab, set **Move** to 1 stud and **Rotate** to 15°. Clean, aligned builds are much easier to script later.

TRY IT

- Build a simple spawn area: a floor, four low walls, and a **SpawnLocation** (Model tab → Spawn).
- Group the walls: select them, press **Ctrl+G**, and name the model `SpawnWalls`.

1.5 Playtesting and your first private publish

Playtest:

- **Play (F5)** spawns you as a character in the game.
- **Stop (Shift+F5)** ends the test.
- **Run (F8)** runs the game without a character, which is useful for watching physics.

Anything you change *during* a playtest is thrown away when you stop. Make edits after stopping.

Publish:

1. **File** → **Publish to Roblox**.
2. Choose **Create new game**, name it `Coin Rush (dev)` and add a short description.
3. After publishing, open **File** → **Game Settings** → **Permissions** (or the experience settings on the Creator Hub) and keep it **Private** while you build.

Now your place is saved in the cloud. You can open it from any computer, and you can join it from your phone to test mobile controls.

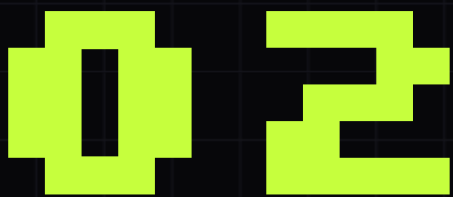
WATCH OUT **Ctrl+S** in a published place saves to Roblox. Use **File** → **Save to File** now and then as an extra local backup.

Module recap

- ☐ Studio installed and 2-step verification enabled
- ☐ You can find Explorer, Properties and Toolbox, and you know what Workspace and ServerScriptService are for
- ☐ You can fly the camera and place, scale and rotate parts with snapping
- ☐ Your spawn area is built and grouped

- ☐ `Coin Rush (dev)` is published as a private place

MODULE 02 OF 09



World Building

YOU'LL SHIP

The complete map for Coin Rush.

- 2.1 Parts, size, color and materials
- 2.2 Anchoring and physics
- 2.3 Terrain and water
- 2.4 Lighting and atmosphere
- 2.5 Using models safely (no backdoors)

A good map does half of your game design for you. It tells players where to go, what's dangerous, and what's worth reaching. In this module you'll build the Coin Rush arena: platforms, a lava floor, terrain around it, and lighting that makes it look finished.

2.1 Parts, size, color and materials

Parts come in a few shapes: **Block**, **Sphere**, **Cylinder**, **Wedge** and **Corner Wedge** (Home tab → arrow under Part). Most maps are 90% blocks and wedges.

Properties you'll use constantly:

Property	What it does
Size	Width, height, depth in studs (1 stud ≈ 28 cm).
Position / Orientation	Where it is and how it's rotated.
Color	Any RGB color.
Material	Surface look and sound: Plastic, SmoothPlastic, Neon, Wood, Slate, Glass...
Transparency	0 = solid, 1 = invisible.
CanCollide	Whether players and objects bump into it.
CastShadow	Turn off for tiny decorative parts to save performance.

Readable design rules for Coin Rush:

- **Safe ground** is calm and matte: SmoothPlastic or Slate in greys and blues.
- **Danger** is loud: bright orange/red **Neon** for lava.
- **Rewards** glow: yellow **Neon** coins.

Players learn these rules in seconds, without any tutorial text.

Try it

Build the arena floor: a 120×2×120 lava plate (Neon, orange), and eight to ten stone platforms of different heights floating above it. Leave gaps a player can jump (up to about 10 studs wide and 7 studs up with default settings).

2.2 Anchoring and physics

Every part is simulated by the physics engine unless it's **Anchored**. An unanchored platform will fall into the void the moment you press Play.

- **Anchored = true**: the part never moves on its own. Use this for all map geometry.
- **Anchored = false**: gravity and collisions apply. Use this for things that should tumble, like crates.

To anchor everything at once: select the parts (or a whole model) and tick **Anchored** in Properties, or use the **Anchor** button in the Home tab.

Welds hold unanchored parts together. If you build a moving object (like a swinging hammer later), weld its parts with **WeldConstraint** and anchor only what should stay still.

WATCH OUT If a playtest "explodes" your build, something important isn't anchored. Stop, select all, anchor, test again.

2.3 Terrain and water

Terrain gives you natural ground, mountains and water that parts can't easily fake.

1. Open the **Terrain Editor** (Home tab → Editor).
2. Use **Generate** for a quick landscape, or **Add / Subtract / Grow / Smooth** with brushes.
3. **Paint** changes the material: Grass, Rock, Sand, Snow, Water...
4. Use **Sea Level** to add water across an area.

For Coin Rush, surround the arena with rocky cliffs so the world feels enclosed and players look *inward* at the action. Keep terrain away from the playable area so it doesn't interfere with lava and platforms.

TIP Terrain is heavier to render than simple parts. Use it for backgrounds and big natural shapes, and keep gameplay surfaces as parts.

2.4 Lighting and atmosphere

Lighting is the fastest way to make a map look professional. Select **Lighting** in the Explorer:

- **ClockTime** : time of day (try 17.5 for a warm sunset).
- **Brightness** , **Ambient** , **OutdoorAmbient** : overall light and shadow color.
- **Technology** : **Future** gives the best shadows and light from Neon parts.

Add these objects inside **Lighting** for instant mood:

Object	Use
Atmosphere	Haze and depth. Density 0.3–0.4 feels cinematic.
Sky	Custom skybox.
BloomEffect	Makes Neon glow. Keep Intensity low (0.5–1).
ColorCorrectionEffect	Tint and contrast. Small changes go far.
SunRaysEffect	Light shafts through gaps.

A good Coin Rush look: sunset clock time, orange-pink atmosphere, subtle bloom so the lava and coins glow.

TIP Test your lighting on a phone. Very dark scenes look fine on a monitor and unplayable on a bright mobile screen.

2.5 Using models safely (no backdoors)

The Toolbox saves time, but free models are also the #1 way games get hacked. Some models contain hidden scripts (called *backdoors*) that let a stranger run code in your game.

Before using any model:

1. Prefer assets from verified creators and Roblox itself.
2. Insert it, then expand it in the Explorer and look for any **Script**, **LocalScript** or **ModuleScript** you didn't expect.
3. Delete scripts you don't need. A tree or a rock never needs a script.
4. Be suspicious of scripts that use `require()` with a number, `getfenv`, `loadstring`, or long unreadable strings.
5. Search your whole game: in the Explorer filter box type `ClassName:Script` or use **Find All** (Ctrl+Shift+F) for `require()`.

WATCH OUT A plugin can edit your game too. Only install plugins with many users and good reviews, and remove the ones you don't use.

Module recap

- ☐ Arena built: lava floor, platforms, and gaps players can jump
- ☐ Every map part is anchored
- ☐ Terrain cliffs frame the arena
- ☐ Lighting set: time of day, Atmosphere, Bloom, color correction
- ☐ No unknown scripts from the Toolbox anywhere in the game

MODULE 03 OF 09



Scripting Fundamentals

YOU'LL SHIP

Working lava and vanishing platforms in Coin Rush.

3.1 Script vs LocalScript vs ModuleScript

3.2 Variables and types

3.3 Conditions and loops

3.4 Functions

3.5 Events: build a lava floor

3.6 Tables

Now the world starts reacting. In this module you'll learn Luau, Roblox's scripting language, and use it to make your first interactive hazards: a lava floor that actually hurts, and platforms that disappear.

3.1 Script vs LocalScript vs ModuleScript

Roblox has three kinds of script, and picking the right one is half the battle.

Type	Runs on	Typical place	Use it for
Script	The server	ServerScriptService, or inside a part	Game rules, damage, saving data, rewards
LocalScript	One player's device	StarterPlayerScripts, StarterGui, ReplicatedFirst	UI, camera, input, effects only that player sees
ModuleScript	Whoever <code>require</code> s it	ReplicatedStorage, ServerScriptService	Shared code and settings

A simple rule: **anything that matters, like health, coins, winning or buying, runs on the server.** The client only shows things and sends requests. You'll see why in Module 5.

To create one: right-click **ServerScriptService** → **Insert Object** → **Script**. Rename it right away.

```
print("Coin Rush server started")
```

Press Play and open **View** → **Output**. Your message appears there. The Output window is where every error shows up, so keep it open always.

3.2 Variables and types

A variable is a named box for a value. In Luau you create one with `local`:

```
local playerName = "Alex"      -- string (text)
local coins = 0                -- number
local isAlive = true           -- boolean (true/false)
local nothing = nil            -- nil means "no value"

coins = coins + 10              -- change it later
print(playerName, "has", coins, "coins")
```

Luau also lets you write types, which helps Studio catch mistakes before you run the game:

```
local speed: number = 16
local title: string = "Coin Rush"
```

Instances are values too. You get them with `game`, `workspace` and `script`:

```
local lava = workspace.WaitForChild("LavaFloor")
lava.Color = Color3.fromRGB(255, 90, 20)
lava.Transparency = 0.1
```

TIP Use `WaitForChild` for things that might not have loaded yet. It waits until the object exists instead of erroring.

3.3 Conditions and loops

Conditions choose what happens:

```
local coins = 120

if coins >= 100 then
    print("You can buy the speed upgrade!")
elseif coins >= 50 then
    print("Almost there.")
else
    print("Keep collecting.")
end
```

Comparison operators: `==` equal, `~=` not equal, `<`, `>`, `<=`, `>=`. Combine with `and`, `or`, `not`.

Loops repeat things:

```
-- Count down from 3
for i = 3, 1, -1 do
    print(i)
    task.wait(1)
end
print("GO!")

-- Repeat while a condition is true
local timeLeft = 10
while timeLeft > 0 do
    timeLeft -= 1
    task.wait(1)
end
```

`task.wait(seconds)` pauses the script. Always put a wait inside a `while true do` loop, or Studio will freeze.



Functions

A function is a reusable block of code with a name:

```
local function formatCoins(amount: number): string
    return amount .. " coins"
end

print(formatCoins(25)) --> 25 coins
```

Functions keep your code short and readable. If you write the same three lines twice, turn them into a function.

```
local function setPlatformVisible(platform: BasePart, visible: boolean)
    platform.Transparency = if visible then 0 else 0.8
    platform.CanCollide = visible
end
```



Events: build a lava floor

Events are how Roblox tells your script that *something happened*: a part was touched, a player joined, a button was clicked. You `Connect` a function to an event, and Roblox calls it every time.

Put this **Script** inside `LavaFloor` :

```

local lava = script.Parent

lava.Touched:Connect(function(hit: BasePart)
    local character = hit.Parent
    local humanoid = character and character:FindFirstChildOfClass("Humanoid")
    if humanoid then
        humanoid.Health = 0
    end
end)

```

What's happening:

1. `Touched` fires whenever any part touches the lava.
2. `hit` is the part that touched it, like a player's foot.
3. The foot's parent is the character model. If it has a **Humanoid**, it's a player (or NPC).
4. Setting `Health = 0` knocks them out, and they respawn.

Vanishing platforms. Put this Script in a platform. It disappears shortly after being touched, then comes back:

```

local platform = script.Parent
local busy = false -- a "debounce": ignore touches while we're already running

platform.Touched:Connect(function(hit)
    if busy then return end
    if not hit.Parent:FindFirstChildOfClass("Humanoid") then return end
    busy = true

    task.wait(0.4)
    platform.Transparency = 0.8
    platform.CanCollide = false

    task.wait(2)
    platform.Transparency = 0
    platform.CanCollide = true
    busy = false
end)

```

WATCH OUT `Touched` fires many times per second while a player stands on a part. Without the `busy` debounce, the code would start dozens of times at once.

3.6 Tables

Tables hold many values. They're Luau's lists *and* dictionaries.

```
-- A list (array)
local colors = { "Red", "Blue", "Green" }
print(colors[1])      --> Red (lists start at 1)
print(#colors)        --> 3

for index, color in colors do
    print(index, color)
end

-- A dictionary
local upgradePrices = {
    Speed = 100,
    Jump = 150,
    Magnet = 300,
}
print(upgradePrices.Speed)    --> 100
upgradePrices.Shield = 500    -- add a new key

for name, price in upgradePrices do
    print(name, "costs", price)
end
```

Settings module. Put all Coin Rush tuning values in one ModuleScript in ReplicatedStorage called `GameConfig`:

```
local GameConfig = {
    RoundLength = 90,      -- seconds
    IntermissionLength = 15,
    CoinValue = 1,
    LavaDamage = 100,
    UpgradePrices = {
        Speed = 100,
        Jump = 150,
    },
}

return GameConfig
```

Any script can now read it:

```
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local GameConfig = require(ReplicatedStorage:WaitForChild("GameConfig"))

print("Rounds last", GameConfig.RoundLength, "seconds")
```

Changing the balance of your whole game now means editing one file.

TRY IT

1. Make three vanishing platforms with different timings (read the timings from `GameConfig`).
2. Change the lava so it only removes 40 health per touch, with a one-second debounce per player. (Hint: store the last hit time per character in a table.)

Module recap

- ☐ You know which script type runs where
- ☐ You can use variables, `if`, `for`, `while` and functions
- ☐ The lava floor knocks players out
- ☐ Vanishing platforms work with a debounce
- ☐ `GameConfig` ModuleScript holds your tuning values



Player & UI

YOU'LL SHIP

An intro screen, a main menu and an in-game HUD.

- 4.1 Build a studio-grade intro screen
- 4.2 ScreenGui, Frames and TextLabels
- 4.3 Scaling for desktop, mobile and console
- 4.4 Animation with TweenService
- 4.5 Main menu and HUD

UI is the first thing players judge. A clean intro, a readable HUD and a menu that works on a phone make a game feel trustworthy before anyone has played a single round. In this module you'll build all three for Coin Rush.

1 Build a studio-grade intro screen

This is the free preview lesson, in short. (The full step-by-step version with explanations is in [free-lesson-intro-screen.md](#).)

Put a **LocalScript** named `IntroSplash` in **ReplicatedFirst**, which is the first thing that runs when a player joins:

```

local ReplicatedFirst = game:GetService("ReplicatedFirst")
local Players = game:GetService("Players")
local TweenService = game:GetService("TweenService")

ReplicatedFirst:RemoveDefaultLoadingScreen()

local playerGui = Players.LocalPlayer:WaitForChild("PlayerGui")

local gui = Instance.new("ScreenGui")
gui.IgnoreGuiInset = true
gui.DisplayOrder = 1000
gui.ResetOnSpawn = false

local bg = Instance.new("Frame")
bg.Size = UDim2.fromScale(1, 1)
bg.BackgroundColor3 = Color3.new(0, 0, 0)
bg.BorderSizePixel = 0
bg.Parent = gui

local logo = Instance.new("ImageLabel")
logo.Image = "rbxassetid://YOUR_ASSET_ID"
logo.BackgroundTransparency = 1
logo.AnchorPoint = Vector2.new(0.5, 0.5)
logo.Position = UDim2.fromScale(0.5, 0.5)
logo.Size = UDim2.fromScale(0.4, 0.4)
logo.ImageTransparency = 1
logo.Parent = bg
Instance.new("UIAspectRatioConstraint", logo).AspectRatio = 1
local scale = Instance.new("UIScale", logo)
scale.Scale = 0.9

gui.Parent = playerGui

TweenService:Create(scale, TweenInfo.new(0.25, Enum.EasingStyle.Back), { Scale = 1
}):Play()
local fadeIn = TweenService:Create(logo, TweenInfo.new(0.25), { ImageTransparency = 0 })
fadeIn:Play()
fadeIn.Completed:Wait()
task.wait(1.75)

local out = TweenInfo.new(0.4)
TweenService:Create(logo, out, { ImageTransparency = 1 }):Play()
local bgOut = TweenService:Create(bg, out, { BackgroundTransparency = 1 })

```

```
bgOut:Play()
bgOut.Completed:Wait()
gui:Destroy()
```

4.2 ScreenGui, Frames and TextLabels

All 2D interface lives inside a **ScreenGui**. You design UI in **StarterGui**; when a player spawns, Roblox copies it into their **PlayerGui**.

The building blocks:

Class	Use
Frame	A rectangle, used as a panel or container.
TextLabel	Text that players read.
TextButton / ImageButton	Things players click or tap.
ImageLabel	Pictures and icons.
UICorner	Rounded corners.
UIStroke	Outlines, for text or frames.
UIListLayout / UIGridLayout	Automatic arrangement of children.
UIPadding	Inner spacing.

Build the HUD layout in StarterGui:

```
StarterGui
├─ HUD (ScreenGui)
│   ├── CoinPanel (Frame, top-left)
│   │   ├── UICorner
│   │   ├── Icon (ImageLabel)
│   │   └─ CoinText (TextLabel) "0"
│   └─ TimerPanel (Frame, top-center)
│       ├── UICorner
│       └─ TimerText (TextLabel) "Intermission"
```

Text tips: use a bold, readable font (**FontFace** → Gotham/Montserrat/Fredoka One style), turn on **TextScaled** with a **UITextSizeConstraint** (MaxTextSize ~36), and add a **UIStroke** so text stays readable on bright backgrounds.

Scaling for desktop, mobile and console

More than half of Roblox players are on phones. UI that looks perfect on your monitor can be unusable on a phone.

Rule 1: use Scale, not Offset. A `UDim2` has four numbers: `(xScale, xOffset, yScale, yOffset)`. Scale is a fraction of the screen; Offset is pixels.

```
UDim2.new(0.2, 0, 0.08, 0)    -- 20% of width, 8% of height: works everywhere
UDim2.new(0, 300, 0, 60)     -- 300x60 px: huge on phones, tiny on 4K
```

Rule 2: lock shapes with `UIAspectRatioConstraint`. A coin icon should stay round on every screen.

Rule 3: position with `AnchorPoint`. `AnchorPoint = (0.5, 0)` plus `Position = (0.5, 0, 0.02, 0)` keeps the timer centered at the top on any screen.

Rule 4: respect the safe area. Phones have notches and Roblox has its top-bar buttons. Keep important UI a little away from the edges, or check `ScreenGui.ScreenInsets`.

Test it: in the **Test** tab open the **Device emulator** and switch between phone, tablet and console sizes. Fix anything that overlaps or gets too small.

Animation with TweenService

`TweenService` smoothly animates any property from its current value to a target.

```
local TweenService = game:GetService("TweenService")

local info = TweenInfo.new(
    0.3,                                -- duration (seconds)
    Enum.EasingStyle.Quad,              -- shape of the motion
    Enum.EasingDirection.Out            -- slow down at the end
)

local tween = TweenService:Create(frame, info, {
    Position = UDim2.fromScale(0.5, 0.5),
    BackgroundTransparency = 0,
})
tween:Play()
```

Juice for the coin counter. Make it "pop" whenever the number changes:

```

local function pop(label: TextLabel)
    local uiScale = label:FindFirstChildOfClass("UIScale") or Instance.new("UIScale",
label)
    uiScale.Scale = 1.25
    TweenService:Create(uiScale, TweenInfo.new(0.25, Enum.EasingStyle.Back), { Scale = 1
}):Play()
end

```

Easing styles change the feeling: `Quad` is calm, `Back` overshoots playfully, `Bounce` is cartoony, `Linear` feels mechanical. Keep UI animations short (0.15–0.4 s). Slow UI feels laggy.

4.5 Main menu and HUD

Main menu. A full-screen frame with the game title, a big **PLAY** button, and a **SHOP** button (wired up in Module 7). Put a LocalScript in the menu ScreenGui:

```

local TweenService = game:GetService("TweenService")

local menu = script.Parent:WaitForChild("Menu")
local playButton = menu:WaitForChild("PlayButton")

playButton.Activated:Connect(function()
    local tween = TweenService:Create(menu, TweenInfo.new(0.3), { Position =
UDim2.fromScale(0, -1) })
    tween:Play()
    tween.Completed:Wait()
    menu.Visible = false
end)

```

Use `Activated` instead of `MouseButton1Click`. It works for mouse, touch and gamepad.

HUD coin counter. This LocalScript in `HUD` shows the player's coins. The `leaderstats` it reads are created in Module 6; until then it simply shows 0.

```

local Players = game:GetService("Players")
local TweenService = game:GetService("TweenService")

local player = Players.LocalPlayer
local coinText = script.Parent:WaitForChild("CoinPanel"):WaitForChild("CoinText")

local function pop()
    local s = coinText:FindFirstChildOfClass("UIScale") or Instance.new("UIScale",
coinText)
    s.Scale = 1.25
    TweenService:Create(s, TweenInfo.new(0.25, Enum.EasingStyle.Back), { Scale = 1
}):Play()
end

local leaderstats = player:WaitForChild("leaderstats")
local coins = leaderstats:WaitForChild("Coins")

coinText.Text = tostring(coins.Value)
coins.Changed:Connect(function(value)
    coinText.Text = tostring(value)
    pop()
end)

```

Round timer. In Module 7 the server will store the round state in attributes on `ReplicatedStorage`. The HUD just listens:

```

local ReplicatedStorage = game:GetService("ReplicatedStorage")
local timerText = script.Parent:WaitForChild("TimerPanel"):WaitForChild("TimerText")

local function refresh()
    local status = ReplicatedStorage:GetAttribute("RoundStatus") or "Waiting"
    local timeLeft = ReplicatedStorage:GetAttribute("TimeLeft") or 0
    timerText.Text = if timeLeft > 0 then `{status} · {timeLeft}s` else status
end

ReplicatedStorage:GetAttributeChangedSignal("RoundStatus"):Connect(refresh)
ReplicatedStorage:GetAttributeChangedSignal("TimeLeft"):Connect(refresh)
refresh()

```

TIP The backtick string ``{status} · {timeLeft}s`` is Luau string interpolation, a clean way to build text from values.

TRY IT

1. Add a subtle idle animation to the PLAY button (scale 1 → 1.05 and back, repeating). Hint: `TweenInfo.new(0.8, Enum.EasingStyle.Sine, Enum.EasingDirection.InOut, -1, true)`.
2. Test the menu and HUD in the device emulator at phone size.

Module recap

- ☐ Intro screen runs once on join
- ☐ HUD with coin counter and round timer, built with Scale and AnchorPoint
- ☐ Main menu with PLAY that animates away
- ☐ Everything checked in the device emulator

MODULE 05 OF 09

05

Client & Server

YOU'LL SHIP A secure pipeline for player actions in Coin Rush.

- 5.1 How the client-server model works
- 5.2 RemoteEvents and RemoteFunctions
- 5.3 Never trust the client
- 5.4 Hardening your game against exploiters

This is the module that separates hobby projects from real games. Every Roblox game is multiplayer, which means code runs in two places at once. Get this right and your game is fair, secure and bug-free. Get it wrong and exploiters will give themselves a million coins on day one.

5.1 How the client-server model works

When someone plays your game:

- **The server** is a Roblox computer in a data center. There's one per game instance. It's the source of truth: coins, health, rounds, purchases.
- **The client** is each player's device. It renders the world, plays sounds, shows UI and reads input.

Replication keeps them in sync. When the server changes something in `Workspace`, like moving a part or spawning a coin, every client sees it. When a client changes something, it usually stays on that client only. That's intentional: it stops one player from changing the world for everyone.

Lives on	Examples
Server only	<code>ServerScriptService</code> , <code>ServerStorage</code> , <code>DataStores</code> , purchases
Client only	<code>PlayerGui</code> , the camera, input, <code>LocalScripts</code>
Both	<code>Workspace</code> , <code>ReplicatedStorage</code>

One notable exception: a player's **own character** movement is controlled by their client (so movement feels instant), then replicated. That's why speed hacks exist, and why the server must sanity-check.

5.2 RemoteEvents and RemoteFunctions

When the client and server need to talk, they use remotes. Create them in `ReplicatedStorage` so both sides can see them.

RemoteEvent: a one-way message.

```
-- ReplicatedStorage/Remotes/BuyUpgrade (RemoteEvent)

-- Client (LocalScript): ask the server to buy something
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local BuyUpgrade = ReplicatedStorage.Remotes.BuyUpgrade

shopButton.Activated:Connect(function()
    BuyUpgrade:FireServer("Speed")
end)
```

```
-- Server (Script in ServerScriptService): handle the request
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local BuyUpgrade = ReplicatedStorage.Remotes.BuyUpgrade

BuyUpgrade.OnServerEvent:Connect(function(player, upgradeName)
    print(player.Name, "wants to buy", upgradeName)
end)
```

Notice that the server receives `player` as the first argument **automatically**. The client can't fake who they are.

The server can message clients too:

```
-- Server → one player
Notify:FireClient(player, "Round starting!")

-- Server → everyone
Notify:FireAllClients("Round starting!")
```

RemoteFunction: a request that waits for an answer.

```
-- Client
local price = GetPrice:InvokeServer("Speed")

-- Server
GetPrice.OnServerInvoke = function(player, upgradeName)
    return GameConfig.UpgradePrices[upgradeName]
end
```

WATCH OUT Never use `InvokeClient` from the server. If the client never answers, the server script hangs forever.

5.3 Never trust the client

Here's the golden rule of Roblox development: **the client can send anything**. Exploiters can fire any `RemoteEvent`, with any arguments, as often as they like.

This code is a disaster:

```
-- ✗ DON'T: the client decides how many coins it gets
AddCoins.OnServerEvent:Connect(function(player, amount)
    player.leaderstats.Coins.Value += amount
end)
```

An exploiter just calls `AddCoins:FireServer(999999)`.

The fix is to send **intent**, not results. The client says *what it wants to do*; the server checks whether it's allowed and decides the outcome:

```
-- ✓ DO: the client asks, the server decides
BuyUpgrade.OnServerEvent:Connect(function(player, upgradeName)
    -- 1. Validate the type and the value
    if typeof(upgradeName) ~= "string" then return end
    local price = GameConfig.UpgradePrices[upgradeName]
    if not price then return end

    -- 2. Check the rules on the server
    local coins = player.leaderstats.Coins
    if coins.Value < price then return end
    if player:GetAttribute("Has" .. upgradeName) then return end

    -- 3. Apply the result on the server
    coins.Value -= price
    player:SetAttribute("Has" .. upgradeName, true)
end)
```

The same goes for collecting coins: the server should detect the touch and award the coin. The client should never say "I collected a coin".

5.4 Hardening your game against exploiters

A checklist to run on every remote:

- 1. Validate types** with `typeof()`. Reject anything unexpected.
- 2. Validate ranges.** Numbers can be negative, huge, `NaN` or `math.huge`. Clamp or reject.
- 3. Check permissions and state.** Can this player do this *right now*? (Alive? In the round? Enough coins?)
- 4. Rate-limit.** Ignore requests that come too fast:

```

local lastRequest: { [Player]: number } = {}

local function allowed(player: Player, cooldown: number): boolean
    local now = os.clock()
    if lastRequest[player] and now - lastRequest[player] < cooldown then
        return false
    end
    lastRequest[player] = now
    return true
end

game:GetService("Players").PlayerRemoving:Connect(function(player)
    lastRequest[player] = nil -- avoid memory leaks
end)

```

5. **Sanity-check movement.** For Coin Rush, when a player collects a coin, check they're actually close to it:

```

local function isNear(player: Player, part: BasePart, maxDistance: number): boolean
    local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
    return root ~= nil and (root.Position - part.Position).Magnitude <= maxDistance
end

```

6. **Keep secrets on the server.** Don't put prices you enforce, loot tables, or admin lists in ReplicatedStorage and trust them from the client. Put server-only code and assets in ServerScriptService and ServerStorage.

TIP A good test: imagine the player's LocalScripts were rewritten by the worst person on the internet. Is your game still fair? If yes, you're done.

TRY IT

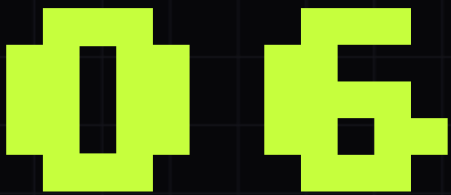
1. Create `ReplicatedStorage/Remotes` with a `BuyUpgrade` RemoteEvent and implement the secure handler above.
2. Add the rate limiter so a player can only attempt one purchase every 0.5 seconds.

Module recap

- ☐ You can explain what runs on the server vs. the client
- ☐ You can send messages both ways with RemoteEvents

- ☐ Every remote validates types, values, state and rate
- ☐ Clients send intent; the server decides results

MODULE 06 OF 09



Data & Progression

YOU'LL SHIP

Persistent player profiles with coins, XP and levels.

- 6.1 Leaderstats: coins and points
- 6.2 DataStoreService
- 6.3 Saving on leave and on server shutdown
- 6.4 XP and levels

Players come back to games that remember them. In this module you'll give every player a coin balance, save it with DataStores so it survives leaving and server shutdowns, and add XP and levels so there's always a next goal.

6.1 Leaderstats: coins and points

Roblox has a built-in leaderboard. If a player has a folder named exactly `leaderstats` with value objects inside, they show up in the top-right player list automatically.

Script in **ServerScriptService** named `PlayerData` :

```
local Players = game:GetService("Players")

Players.PlayerAdded:Connect(function(player)
    local leaderstats = Instance.new("Folder")
    leaderstats.Name = "leaderstats"
    leaderstats.Parent = player

    local coins = Instance.new("IntValue")
    coins.Name = "Coins"
    coins.Value = 0
    coins.Parent = leaderstats

    local level = Instance.new("IntValue")
    level.Name = "Level"
    level.Value = 1
    level.Parent = leaderstats
end)
```

Press Play: your name appears with Coins and Level. The HUD from Module 4 now shows your coin count too.

WATCH OUT Only the **server** should change these values. If a LocalScript changes them, only that player sees the change, and it won't save.

6.2 DataStoreService

A **DataStore** is a cloud database Roblox gives every game for free. You save a value under a key (like the player's UserId) and load it next time.

First, allow Studio to use it: **File** → **Game Settings** → **Security** → **Enable Studio Access to API Services**. The place must be published.

DataStore calls go over the network and **can fail**, so every call is wrapped in `pcall` :

```
local DataStoreService = game:GetService("DataStoreService")
local store = DataStoreService:GetDataStore("PlayerData_v1")

local ok, data = pcall(function()
    return store:GetAsync("Player_" .. player.UserId)
end)

if ok then
    -- data is the saved table, or nil for a brand-new player
else
    warn("Load failed:", data)
end
```

Save a **table** per player, not separate keys per stat. One read and one write per player is faster and stays within limits:

```
local DEFAULT_DATA = {
    Coins = 0,
    XP = 0,
    Level = 1,
    Upgrades = {},
}
```

6.3 Saving on leave and on server shutdown

Here's the complete, production-style `PlayerData` script. It loads with retries, keeps a session copy in memory, saves when a player leaves, autosaves every few minutes, and saves everyone when the server shuts down.

```

local Players = game:GetService("Players")
local DataStoreService = game:GetService("DataStoreService")

local store = DataStoreService:GetDataStore("PlayerData_v1")

local DEFAULT_DATA = { Coins = 0, XP = 0, Level = 1, Upgrades = {} }
local AUTOSAVE_INTERVAL = 180

local sessionData: { [Player]: any } = {}

local function keyFor(player: Player): string
    return "Player_" .. player.UserId
end

local function withRetries<T>(fn: () -> T, attempts: number): (boolean, T?)
    for attempt = 1, attempts do
        local ok, result = pcall(fn)
        if ok then return true, result end
        warn(`DataStore attempt {attempt} failed: {result}`)
        task.wait(2 ^ attempt)
    end
    return false, nil
end

local function load(player: Player)
    local ok, saved = withRetries(function()
        return store:GetAsync(keyFor(player))
    end, 3)

    if not ok then
        -- Don't let them play with a blank profile that would overwrite real data.
        player:Kick("Couldn't load your data. Please rejoin in a minute.")
        return
    end

    local data = table.clone(DEFAULT_DATA)
    data.Upgrades = {}
    if type(saved) == "table" then
        for k, v in saved do data[k] = v end
    end
    sessionData[player] = data

    local leaderstats = Instance.new("Folder")

```

```

    leaderstats.Name = "leaderstats"
    local coins = Instance.new("IntValue")
    coins.Name = "Coins"
    coins.Value = data.Coins
    coins.Parent = leaderstats
    local level = Instance.new("IntValue")
    level.Name = "Level"
    level.Value = data.Level
    level.Parent = leaderstats
    leaderstats.Parent = player

    -- Keep the session copy in sync with the visible values.
    coins.Changed:Connect(function(v) data.Coins = v end)
    level.Changed:Connect(function(v) data.Level = v end)
end

local function save(player: Player)
    local data = sessionData[player]
    if not data then return end
    withRetries(function()
        return store.UpdateAsync(keyFor(player), function()
            return data
        end)
    end, 3)
end

Players.PlayerAdded:Connect(load)
for _, p in Players:GetPlayers() do task.spawn(load, p) end

Players.PlayerRemoving:Connect(function(player)
    save(player)
    sessionData[player] = nil
end)

-- Autosave
task.spawn(function()
    while true do
        task.wait(AUTOSAVE_INTERVAL)
        for player in sessionData do
            task.spawn(save, player)
        end
    end
end)
end)

```

```
-- Server shutdown: save everyone before the server closes (max ~30 s)
game:BindToClose(function()
    local pending = 0
    for player in sessionData do
        pending += 1
        task.spawn(function()
            save(player)
            pending -= 1
        end)
    end
    while pending > 0 do task.wait() end
end)
```

Why each part matters:

- **Retries with backoff:** DataStores occasionally fail; waiting 2, 4, 8 seconds and retrying fixes most failures.
- **Kick on failed load:** if you let a player in with default data, the next save overwrites their real progress. Kicking is annoying, but losing progress is far worse.
- `UpdateAsync` instead of `SetAsync` : safer when several servers might write the same key.
- `BindToClose` : when Roblox shuts a server down (for example after you publish an update), players don't get `PlayerRemoving` in time without it.

TIP Changing the store name (`PlayerData_v1` → `PlayerData_v2`) gives everyone a fresh start. Use it for testing, never casually in a live game.



XP and levels

Levels give players a reason to play one more round. Use a formula so each level needs a bit more XP than the last:

```
local function xpForLevel(level: number): number
    return math.floor(50 * level ^ 1.5)
end

-- Level 1→2: 50 XP, 2→3: 141, 5→6: 559, 10→11: 1581
```

A server function to award XP, used by the round system in Module 7:

```

local LevelUp = game:GetService("ReplicatedStorage").Remotes.LevelUp -- RemoteEvent

local function addXP(player: Player, amount: number)
    local data = sessionData[player]
    if not data then return end
    data.XP += amount
    while data.XP >= xpForLevel(data.Level) do
        data.XP -= xpForLevel(data.Level)
        data.Level += 1
        player.leaderstats.Level.Value = data.Level
        LevelUp:FireClient(player, data.Level)
    end
end
end

```

On the client, listen to `LevelUp` and play a big celebratory UI animation, a sound, and a particle burst. That moment of feedback is what makes progression feel good.

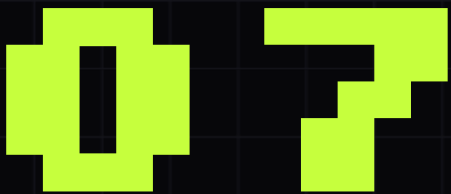
Sharing functions between scripts. Other server scripts (the round system, the shop) need `addXP` and access to coins. Turn the data code into a **ModuleScript** `PlayerDataService` in `ServerScriptService` that returns a table with `get(player)`, `addCoins(player, n)` and `addXP(player, n)`, and have one Script `require` it on startup. That's the structure Module 7 uses.

TRY IT

1. Collect a few coins, stop the test, press Play again: your coins should still be there.
2. Add a `TotalCoinsEver` stat that only goes up, and show it in a stats panel.

Module recap

- ☐ leaderstats show Coins and Level
- ☐ Data loads with retries and kicks on failure instead of overwriting
- ☐ Saves on leave, every 3 minutes, and on server shutdown
- ☐ XP curve and level-ups with a client celebration



The Game Loop

YOU'LL SHIP A complete, playable round-based game.

7.1 Designing your game on paper

7.2 Rounds and timers

7.3 Collectibles and scoring

7.4 Upgrade shop

7.5 Sound and visual effects

Everything comes together here. You'll design Coin Rush on paper, then build the round system, coins that spawn and get collected, an upgrade shop, and the sound and visual effects that make it all feel great.

7.1 Designing your game on paper

Before writing the round script, answer these on one page:

1. **Core loop:** what does the player do over and over? *Jump between platforms collecting coins before the timer ends, without touching lava.*
2. **Session loop:** what does one round look like? *15 s intermission → 90 s round → results → repeat.*
3. **Progression:** why come back tomorrow? *Saved coins, levels, and upgrades that change how you play.*
4. **Win condition:** *Most coins collected this round wins a bonus.*
5. **First 30 seconds:** *Intro splash, menu, PLAY, you're in the lobby watching the current round with a countdown.*

Then list every system you need. For Coin Rush: round manager, coin spawner, coin collection, rewards, shop, effects. Build them in that order, testing after each.

TIP Cut features until the list scares you a little less. A small game that's finished beats a big game that isn't.

7.2 Rounds and timers

The round manager is a single **Script** in ServerScriptService. It publishes its state through **attributes** on ReplicatedStorage, which the HUD from Module 4 already listens to.

Set up the map first: a **Lobby** model with a **LobbySpawn** part, and an **Arena** folder containing two folders: **Spawns** (parts where players start a round) and **CoinSpots** (invisible, anchored parts where coins can appear).

```

-- ServerScriptService/RoundManager
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local ServerScriptService = game:GetService("ServerScriptService")

local GameConfig = require(ReplicatedStorage.GameConfig)
local PlayerDataService = require(ServerScriptService.PlayerDataService)
local CoinSpawner = require(ServerScriptService.CoinSpawner)

local arena = workspace.Arena
local lobbySpawn = workspace.Lobby.LobbySpawn

local function setState(status: string, timeLeft: number)
    ReplicatedStorage:SetAttribute("RoundStatus", status)
    ReplicatedStorage:SetAttribute("TimeLeft", timeLeft)
end

local function teleport(player: Player, target: BasePart)
    local root = player.Character and player.Character:FindFirstChild("HumanoidRootPart")
    if root then
        root.CFrame = target.CFrame + Vector3.new(0, 4, 0)
    end
end

local function countdown(status: string, seconds: number, stopEarly: (() -> boolean)?)
    for t = seconds, 1, -1 do
        setState(status, t)
        task.wait(1)
        if stopEarly and stopEarly() then return end
    end
end

while true do
    -- Intermission: wait for at least one player
    repeat
        setState("Waiting for players", 0)
        task.wait(1)
    until #Players:GetPlayers() >= 1

    countdown("Intermission", GameConfig.IntermissionLength)

    -- Start the round
    local spawns = arena.Spawns:GetChildren()

```

```

local roundCoins: { [Player]: number } = {}
for i, player in Players:GetPlayers() do
    roundCoins[player] = 0
    teleport(player, spawns[(i - 1) % #spawns + 1])
end

CoinSpawner.start(function(player: Player)
    if roundCoins[player] then
        roundCoins[player] += 1
        PlayerDataService.addCoins(player, GameConfig.CoinValue)
    end
end)

countdown("Round", GameConfig.RoundLength, function()
    return #Players:GetPlayers() == 0
end)

CoinSpawner.stop()

-- Results
local winner, best = nil, -1
for player, count in roundCoins do
    if player.Parent and count > best then
        winner, best = player, count
    end
    if player.Parent then
        PlayerDataService.addXP(player, 10 + count)
    end
end
if winner then
    PlayerDataService.addCoins(winner, GameConfig.WinnerBonus)
    setState(`{winner.DisplayName} wins with {best} coins!`, 0)
else
    setState("Round over", 0)
end
task.wait(5)

for _, player in Players:GetPlayers() do
    teleport(player, lobbySpawn)
end
end

```

Add `WinnerBonus = 25` to `GameConfig`.

`PlayerDataService` is the ModuleScript you built from Module 6. The round manager only needs three functions from it:

```
PlayerDataService.addCoins(player, amount)    -- server-side only
PlayerDataService.addXP(player, amount)
PlayerDataService.trySpend(player, amount)    -- returns true if they could afford it
```

7.3 Collectibles and scoring

Coins are spawned **by the server**, from a template in **ServerStorage** (so clients can't copy or fake them). Build one coin: a yellow Neon cylinder rotated on its side, size about `0.4, 2.5, 2.5`, Anchored, `CanCollide = false`. Name it `Coin` and move it into ServerStorage.

```

-- ServerScriptService/CoinSpawner (ModuleScript)
local ServerStorage = game:GetService("ServerStorage")
local Players = game:GetService("Players")

local template = ServerStorage:WaitForChild("Coin")
local spots = workspace.Arena.CoinSpots:GetChildren()

local CoinSpawner = {}
local running = false
local active: { BasePart } = {}

local function spawnCoin(onCollect: (Player) -> ())
    local spot = spots[math.random(1, #spots)]
    local coin = template:Clone()
    coin.CFrame = spot.CFrame
    coin.Parent = workspace
    table.insert(active, coin)

    local taken = false
    coin.Touched:Connect(function(hit)
        if taken then return end
        local player = Players:GetPlayerFromCharacter(hit.Parent)
        if not player then return end
        taken = true
        onCollect(player)
        coin:Destroy()
    end)
end

function CoinSpawner.start(onCollect: (Player) -> ())
    running = true
    task.spawn(function()
        while running do
            if #active < 25 then
                spawnCoin(onCollect)
            end
            -- Forget coins that were destroyed
            for i = #active, 1, -1 do
                if not active[i].Parent then table.remove(active, i) end
            end
            task.wait(0.5)
        end
    end)
end

```

```

end

function CoinSpawner.stop()
    running = false
    for _, coin in active do coin:Destroy() end
    table.clear(active)
end

return CoinSpawner

```

Because the coin detects the touch **on the server** and uses `GetPlayerFromCharacter`, a client can't claim coins it never touched.

Spinning coins. Spinning 25 coins on the server wastes bandwidth. Spin them on each client instead, in a LocalScript in StarterPlayerScripts:

```

local RunService = game:GetService("RunService")

RunService.RenderStepped:Connect(function(dt)
    for _, coin in workspace:GetChildren() do
        if coin.Name == "Coin" and coin:IsA("BasePart") then
            coin.CFrame *= CFrame.Angles(0, dt * 3, 0)
        end
    end
end)

```

Upgrade shop

Two upgrades for Coin Rush: **Speed** (WalkSpeed 16 → 22) and **Jump** (JumpPower 50 → 60). They're bought with coins and saved in the player's `Upgrades` table.

Server (extends the secure handler from Module 5):

```

-- ServerScriptService/Shop
local Players = game:GetService("Players")
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local ServerScriptService = game:GetService("ServerScriptService")

local GameConfig = require(ReplicatedStorage.GameConfig)
local PlayerDataService = require(ServerScriptService.PlayerDataService)
local BuyUpgrade = ReplicatedStorage.Remotes.BuyUpgrade

local EFFECTS = {
    Speed = function(humanoid: Humanoid) humanoid.WalkSpeed = 22 end,
    Jump = function(humanoid: Humanoid)
        humanoid.UseJumpPower = true
        humanoid.JumpPower = 60
    end,
}

local function applyUpgrades(player: Player, character: Model)
    local humanoid = character:WaitForChild("Humanoid") :: Humanoid
    local data = PlayerDataService.get(player)
    if not data then return end
    for name in data.Upgrades do
        if EFFECTS[name] then EFFECTS[name](humanoid) end
    end
end

Players.PlayerAdded:Connect(function(player)
    player.CharacterAdded:Connect(function(character)
        applyUpgrades(player, character)
    end)
end)

BuyUpgrade.OnServerEvent:Connect(function(player, name)
    if typeof(name) ~= "string" or not EFFECTS[name] then return end
    local data = PlayerDataService.get(player)
    if not data or data.Upgrades[name] then return end
    local price = GameConfig.UpgradePrices[name]
    if not PlayerDataService.trySpend(player, price) then return end

    data.Upgrades[name] = true
    if player.Character then applyUpgrades(player, player.Character) end
end)

```

Client: the SHOP button from the main menu opens a panel with one button per upgrade, each calling `BuyUpgrade:FireServer("Speed")`. Show the price from `GameConfig`, and grey out owned upgrades by reading an attribute or a RemoteFunction that returns the player's upgrades.

7.5 Sound and visual effects

Feedback turns "it works" into "it feels amazing". Add these:

Coin pickup (client-side, so it's instant). The server tells the collecting player with a RemoteEvent `CoinCollected`, passing the coin's position:

```
-- In CoinSpawner, right before coin:Destroy():  
ReplicatedStorage.Remotes.CoinCollected:FireClient(player, coin.Position)
```

```

-- StarterPlayerScripts/CoinEffects (LocalScript)
local ReplicatedStorage = game:GetService("ReplicatedStorage")
local SoundService = game:GetService("SoundService")
local Debris = game:GetService("Debris")

local pickupSound = SoundService.WaitForChild("CoinPickup")

ReplicatedStorage.Remotes.CoinCollected.OnClientEvent:Connect(function(position: Vector3)
    pickupSound.PlaybackSpeed = 0.95 + math.random() * 0.15 -- small variation feels
    natural
    SoundService.PlayLocalSound(pickupSound)

    local burst = Instance.new("Part")
    burst.Anchored, burst.CanCollide, burst.Transparency = true, false, 1
    burst.Position = position
    burst.Parent = workspace

    local sparks = Instance.new("ParticleEmitter")
    sparks.Color = ColorSequence.new(Color3.fromRGB(255, 220, 60))
    sparks.LightEmission = 1
    sparks.Speed = NumberRange.new(8, 14)
    sparks.Lifetime = NumberRange.new(0.3, 0.6)
    sparks.SpreadAngle = Vector2.new(180, 180)
    sparks.Size = NumberSequence.new(0.4, 0)
    sparks.Parent = burst
    sparks:Emit(20)

    Debris:AddItem(burst, 1)
end)

```

More juice to add:

- A countdown beep for the last 5 seconds of a round, and a horn when it ends.
- Lava bubbling: a `ParticleEmitter` on the lava floor plus a low looping sound.
- Camera shake or a red screen flash when a player falls in lava.
- Background music in lobby vs. round (two `Sound` objects in `SoundService`, crossfaded with `TweenService` on `Volume`).

Find sounds in the Toolbox **Audio** tab or the Creator Store. Use Roblox-provided or properly licensed audio only.

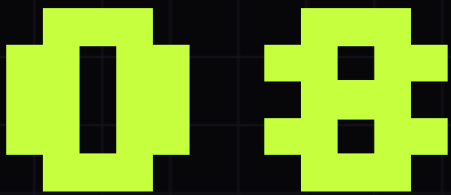
TRY IT

1. Play a full round with a friend (or with **Test** → **Clients and Servers** → **2 players**).
2. Tune `RoundLength`, coin count and upgrade prices in `GameConfig` until rounds feel tense but fair.

Module recap

- ☐ One-page design with core loop, session loop and progression
- ☐ RoundManager cycles intermission → round → results
- ☐ Server-spawned coins, collected securely, with a winner bonus
- ☐ Speed and Jump upgrades bought with coins and saved
- ☐ Pickup sound and particles, plus at least two other effects

MODULE 08 OF 09



Monetization

YOU'LL SHIP A working in-game store.

- 8.1 Game Passes
- 8.2 Developer Products
- 8.3 Bulletproof ProcessReceipt
- 8.4 Pricing that doesn't push players away

Roblox games earn Robux through two kinds of purchases: **Game Passes** (buy once, own forever) and **Developer Products** (buy as many times as you like). In this module you'll add both to Coin Rush, handle purchases without ever losing a player's Robux, and price them so players feel good about paying.

8.1 Game Passes

A Game Pass is a permanent perk. For Coin Rush: **2× Coins**.

Create it:

1. Open create.roblox.com → **Creations**, select your experience.
2. Go to **Monetization** → **Passes** → **Create a Pass**, and upload an icon (512×512).
3. After creating it, open the pass, set it **For Sale** and choose a price.
4. Copy the **Pass ID** (the number in the URL or on the pass page).

Check ownership and apply the perk (server):

```
local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")

local DOUBLE_COINS_PASS = 0000000000 -- your Pass ID

local function checkPass(player: Player)
    local ok, owns = pcall(function()
        return MarketplaceService:UserOwnsGamePassAsync(player.UserId, DOUBLE_COINS_PASS)
    end)
    player:SetAttribute("DoubleCoins", ok and owns)
end

Players.PlayerAdded:Connect(checkPass)

-- When they buy it during the session
MarketplaceService.PromptGamePassPurchaseFinished:Connect(function(player, passId,
    purchased)
    if purchased and passId == DOUBLE_COINS_PASS then
        player:SetAttribute("DoubleCoins", true)
    end
end)
```

Then in `PlayerDataService.addCoins`, multiply when the attribute is set:

```
if player:GetAttribute("DoubleCoins") then
    amount *= 2
end
```

Prompt the purchase (client):

```
local MarketplaceService = game:GetService("MarketplaceService")
local player = game:GetService("Players").LocalPlayer

buyDoubleCoinsButton.Activated:Connect(function()
    MarketplaceService:PromptGamePassPurchase(player, DOUBLE_COINS_PASS)
end)
```

#.2 Developer Products

Developer Products can be bought repeatedly: coin packs, revives, skip-a-wait. For Coin Rush: **500 Coins**.

Create it under **Monetization** → **Developer Products** → **Create a Developer Product**, set the price, and copy the **Product ID**.

Prompt it from the client:

```
MarketplaceService:PromptProductPurchase(player, COIN_PACK_500)
```

The actual reward is given by the server in `ProcessReceipt`, the most important purchase function in Roblox.

#.3 Bulletproof ProcessReceipt

When a player buys a Developer Product, Roblox calls your `MarketplaceService.ProcessReceipt` callback on the server. You grant the item and return `PurchaseGranted`. If you return `NotProcessedYet` (or the server crashes), Roblox **tries again later**, even in another server. That retry is what protects players' Robux.

Rules:

1. Set `ProcessReceipt` **once**, in **one** server Script.
2. Only return `PurchaseGranted` **after** the reward is actually given and saved.
3. Remember which receipts you already granted, so a retry doesn't give the reward twice.

```

-- ServerScriptService/Purchases
local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")
local DataStoreService = game:GetService("DataStoreService")
local ServerScriptService = game:GetService("ServerScriptService")

local PlayerDataService = require(ServerScriptService.PlayerDataService)
local receipts = DataStoreService:GetDataStore("PurchaseReceipts_v1")

local COIN_PACK_500 = 0000000000 -- your Product ID

local handlers: { [number]: (Player) -> boolean } = {
    [COIN_PACK_500] = function(player)
        PlayerDataService.addCoins(player, 500, { ignoreMultiplier = true })
        return true
    end,
}

MarketplaceService.ProcessReceipt = function(receipt)
    local key = `{receipt.PlayerId}_{receipt.PurchaseId}`

    -- Already granted? Tell Roblox we're done.
    local ok, alreadyGranted = pcall(function()
        return receipts:GetAsync(key)
    end)
    if not ok then
        return Enum.ProductPurchaseDecision.NotProcessedYet
    end
    if alreadyGranted then
        return Enum.ProductPurchaseDecision.PurchaseGranted
    end

    -- The player must be in this server to receive the item.
    local player = Players:GetPlayerByUserId(receipt.PlayerId)
    local handler = handlers[receipt.ProductId]
    if not player or not handler then
        return Enum.ProductPurchaseDecision.NotProcessedYet
    end

    local granted = handler(player)
    if not granted then
        return Enum.ProductPurchaseDecision.NotProcessedYet
    end
end

```

```

-- Record the receipt, then save the player's data right away.
local saved = pcall(function()
    receipts:SetAsync(key, true)
end)
if not saved then
    return Enum.ProductPurchaseDecision.NotProcessedYet
end
PlayerDataService.saveNow(player)

return Enum.ProductPurchaseDecision.PurchaseGranted
end

```

Two small additions to `PlayerDataService` make this work: expose the `save` function from Module 6 as `saveNow`, and give `addCoins` an optional `{ ignoreMultiplier = true }` flag so bought coins aren't doubled by the 2× pass.

WATCH OUT Never grant developer products from a `RemoteEvent` or from `PromptProductPurchaseFinished`. Those can fire even when no Robux were paid. `ProcessReceipt` is the only trustworthy signal.

Testing: purchases in Studio are simulated and free. Test every product in Studio, then once more in the live game with a cheap price.

Pricing that doesn't push players away

Money follows goodwill. Players pay when the game already feels generous.

Principles:

- **Fun first, for free.** A free player must be able to enjoy and finish everything. Paid items make it *faster, cooler* or *more convenient*, not possible.
- **Avoid pay-to-win in competitive modes.** In Coin Rush, 2× Coins speeds up progression but doesn't give an edge inside a round.
- **Offer a cheap first purchase.** A 25–50 Robux item gets players over the "first purchase" barrier.
- **Anchor with a bigger bundle.** Show the 500-coin pack next to a 1,500-coin pack with a "best value" label.
- **Show, don't nag.** Place the shop button clearly; never block gameplay with repeated purchase pop-ups.
- **Time offers to moments of desire**, like after a close loss ("Revive for 15 R\$") or when a player has almost enough coins for an upgrade.

Typical price ranges (always test for your game): cosmetic items 25–150 R\$, 2× passes 99–249 R\$, VIP pass 149–499 R\$, currency packs from 25 R\$ upward.

Track results in **Analytics** → **Monetization** (Module 9) and change one price at a time so you know what caused a change.

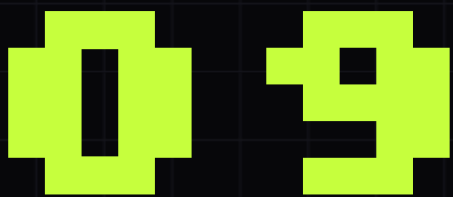
TRY IT

1. Create the 2× Coins pass and the 500 Coins product, and test both in Studio.
2. Add a "best value" 1,500-coin product and see which one testers pick.

Module recap

- ☐ 2× Coins Game Pass works, including when bought mid-session
- ☐ 500 Coins Developer Product granted only in ProcessReceipt
- ☐ Receipts are recorded, so retries never double-grant
- ☐ Store prices follow the fun-first rules

MODULE 09 OF 09



Launch & Growth

YOU'LL SHIP

Coin Rush, live and public.

- 9.1 Icon and thumbnails that get clicks
- 9.2 Title, description and discoverability
- 9.3 Analytics: finding where players drop off
- 9.4 Updates and building a community
- 9.5 Running Roblox Ads

A great game nobody clicks on is invisible. In this final module you'll package Coin Rush so it gets clicked, publish it, read the data to find out where players leave, and build a loop of updates and community that keeps it growing.

9.1 Icon and thumbnails that get clicks

On the Roblox home page, your game is a small square icon and a title. That's all a player sees before deciding. The icon is your most important piece of marketing.

Icon (512×512):

- **One clear subject.** A character mid-action grabbing a glowing coin, lava below. Not a screenshot of the whole map.
- **Readable at thumbnail size.** Zoom out to 15% and check it still reads.
- **High contrast, saturated colors.** Warm lava orange vs. cool blue sky stands out in a grid of icons.
- **Little or no text.** Maybe one word. Text becomes unreadable when small.
- **An emotion.** Faces with a strong expression (excitement, panic) get more clicks.

Thumbnails (1920×1080, up to 10): these appear on the game page and sometimes in discovery. Show real gameplay moments with a short caption: *"Race for coins!"*, *"Don't touch the lava!"*, *"Upgrade your speed!"*. The first thumbnail matters most.

How to make them: pose characters in Studio, use good lighting (Module 2), take high-resolution screenshots, then finish in an image editor with glow, color grading and a bold caption. Keep the style consistent with your icon.

TIP Make 3 icon variants. Roblox lets you run **thumbnail and icon experiments** in the Creator Hub, so let the data pick the winner.

9.2 Title, description and discoverability

Title: short, clear, genre-revealing. *Coin Rush* works. You can add a tag for updates, like *Coin Rush [NEW MAP]*, but don't spam emojis and capitals.

Description: first two lines matter most (they show without expanding):

Race other players to grab the most coins before time runs out,
and don't touch the lava! 🔥💰

★ Level up and unlock Speed and Jump upgrades

★ New rounds every 2 minutes

★ Play with friends on PC, mobile and console

👍 Like the game for more updates!

Settings that affect discovery (File → Game Settings, or the Creator Hub):

- **Genre** set correctly.
- **Supported devices**: turn on phone, tablet and console if your UI works there (it does, after Module 4).
- **Maturity/content questionnaire** filled in honestly; it decides who can see your game.
- **Server size**: 8–12 works well for Coin Rush.

Make it public: in the Creator Hub, set the experience to **Public**. Play it yourself from a phone and a PC before you tell anyone.

9.3

Analytics: finding where players drop off

Roblox gives you detailed analytics in the **Creator Hub** → **your experience** → **Analytics**. The three numbers that matter most at the start:

Metric	What it tells you	Healthy early target
Day 1 retention	% of new players who come back the next day	10%+ is a good start
Average session time	How long a visit lasts	8+ minutes for a round game
Payer conversion	% of players who buy anything	1–3%

Also check the **Funnel** style data: how many players leave in the first minute. If lots of players quit in the first 60 seconds, your intro, lobby or first round is confusing or slow.

Custom events show exactly where players go. From the server:

```

local AnalyticsService = game:GetService("AnalyticsService")

-- Log each step of a new player's first session
AnalyticsService:LogOnboardingFunnelStepEvent(player, 1, "Joined")
AnalyticsService:LogOnboardingFunnelStepEvent(player, 2, "Pressed Play")
AnalyticsService:LogOnboardingFunnelStepEvent(player, 3, "Finished First Round")
AnalyticsService:LogOnboardingFunnelStepEvent(player, 4, "Bought First Upgrade")

```

Now you can see the exact step where most people leave, and fix that one first.

TIP Change one thing at a time and compare a week before vs. a week after. If you change five things at once, you'll never know which one worked.

9.4 Updates and building a community

Games grow through updates. A steady rhythm tells the algorithm and your players that the game is alive.

Update rhythm: a small update every 1–2 weeks (a new map, a new upgrade, a limited-time event), and a bigger one every month or two. Put the update in the title tag and the description.

Changelog: a small in-game "What's new" panel on the menu. Players who see new content stay longer.

Community:

- Create a **Roblox Community** (formerly Groups) for your game, and link it on the game page. Members get notified about updates.
- A **Discord server** for fans: bug reports, ideas, sneak peeks.
- **Short videos:** 15–30 s clips of funny lava fails or close finishes on TikTok, YouTube Shorts and Instagram Reels. This is how most small games get their first thousand players.
- Reward your community: codes for free coins announced on Discord, and credits for players whose ideas you add.

Listen, then decide. Players are great at telling you *what* feels wrong and often bad at proposing the *fix*. Look for patterns in feedback, then design your own solution.

9.5 Running Roblox Ads

Once your analytics look healthy (players stay and come back), paid ads can speed up growth. Before that, ads just buy players who leave.

Ads Manager is in the **Creator Hub** → **Ads**. You can run:

- **Sponsored experiences:** your game shown in discovery spots, paid with an ad budget.
- **Display/portal ads** in other games (depending on current Roblox ad products).

How to start:

1. Start small: a modest daily budget for 3–5 days.
2. Use your best-performing icon from the experiments in 9.1.
3. Watch cost per play and, more importantly, **what those players do**: do they stay and come back like organic players?
4. Only scale budgets for ads that bring players who retain.

WATCH OUT Ads can't fix a game with poor retention. Fix the first 5 minutes first, then pay for traffic.

You did it

Look at what you built: a map, secure scripts, polished UI, saved progression, a full round-based game loop, a store, and a public launch. That's the complete skill set behind almost every successful Roblox game.

Where to go next: pick a Genre Pack and take these same systems into an obby, tycoon, simulator, horror, heist or tower defense game. Keep shipping. Every game you finish makes the next one better.

Module recap

- ☐ Icon and at least 3 thumbnails, with an icon experiment running
- ☐ Clear title and a description with a strong first two lines
- ☐ Coin Rush set to Public on all supported devices
- ☐ Onboarding funnel events logged and checked after a week
- ☐ First update planned, community group and Discord created